CSI3007 - Advanced Python Programming

LAB ASSESSMENT - 2

NAME : RAJAVELMURUGAN S

REG_NO : 23 MID0018

FACULTY : SHARMILA BANU K

LAB SLOT : L31+L32+L49+L50

QUESTION 1 : LONGEST SUBSTRING WITHOUT REPEATING CHARACTERS

3. Longest Substring Without Repeating Characters

Medium | Topics | Companies | Hint

Given a string `s`, find the length of the **longest substring** without duplicate characters.

Example 1:

```
Input: s = "abcabcbb"
Output: 3
Explanation: The answer is "abc", with the length of 3. Note that "bca" and "cab" are
also correct answers.
```

Example 2:

```
Input: s = "bbbbb"
Output: 1
Explanation: The answer is "b", with the length of 1.
```

Example 3:

```
Input: s = "pwwkew"
Output: 3
Explanation: The answer is "wke", with the length of 3.
Notice that the answer must be a substring, "pwke" is a subsequence and not a substring.
```

Constraints:

- `0 <= s.length <= 5 * 10^4`
- `s` consists of English letters, digits, symbols and spaces.

INPUT : Takes a input string s from user

OUTPUT : Longest substring without repeating characters and length of substring

DATA STRUCTURE USED : Dictionary

REASON : Dictionary is used to store the last index of each character (Stores character with index) For example char_map = { ‘a’ :0} a – character 0 – last index of “a”

LOGIC : The program uses the sliding window technique to find the longest substring without repeating characters.It stores the latest index of each character in a dictionary and moves the left pointer when ever a duplicate is found.It continuously updates the maximum length and starting index, then prints the longest substring and its length.

What for loop does in this program :

1. **Traverses each character**
2. **Assigns the current character to current_char**

CODE:

```
def length_of_longest_substring(s: str):
    char_map = {}                    # Stores character positions

    left = 0                         # Left pointer
    max_len = 0                      # Maximum length
    start_idx = 0                    # Start index of answer

    for right in range(len(s)):      # Move right pointer
        current_char = s[right]      # Current character

        if current_char in char_map and char_map[current_char] >= left:
            left = char_map[current_char] + 1   # Shift left pointer

        char_map[current_char] = right      # Update character index

        if right - left + 1 > max_len:
            max_len = right - left + 1      # Update maximum length
            start_idx = left                # Save starting index

    longest_substring = s[start_idx:start_idx + max_len]   # Extract substring

    print("Longest Substring:", longest_substring)        # Display substring

    print("Length:", max_len)                             # Display length

s = input("Enter a string: ")                             # Read input
length_of_longest_substring(s)                            # Call function
```

OUTPUT:

TEST CASE – 1

```
Enter a string:  rajav
Longest Substring: raj
Length: 3
```

TEST CASE – 2

```
Enter a string:  abcabcbb
Longest Substring: abc
Length: 3
```

[8]:
```python
def length_of_longest_substring(s: str):
    char_map = {}                           # Stores character positions

    left = 0                                # Left pointer
    max_len = 0                             # Maximum length
    start_idx = 0                           # Start index of answer

    for right in range(len(s)):             # Move right pointer
        current_char = s[right]             # Current character

        if current_char in char_map and char_map[current_char] >= left:
            left = char_map[current_char] + 1   # Shift left pointer

        char_map[current_char] = right      # Update character index

        if right - left + 1 > max_len:
            max_len = right - left + 1      # Update maximum length
            start_idx = left                # Save starting index

    longest_substring = s[start_idx:start_idx + max_len]   # Extract substring

    print("Longest Substring:", longest_substring)         # Display substring

    print("Length:", max_len)                              # Display length

s = input("Enter a string: ")                              # Read input
length_of_longest_substring(s)                             # Call function
```

```
Enter a string:  rajav
Longest Substring: raj
Length: 3
```

QUESTION 2 : COUNT PRIMES

204. Count Primes

Medium | Topics | Companies | Hint

Given an integer `n`, return *the number of prime numbers that are strictly less than* `n`.

Example 1:

```
Input: n = 10
Output: 4
Explanation: There are 4 prime numbers less than 10, they are 2, 3, 5, 7.
```

Example 2:

```
Input: n = 0
Output: 0
```

Example 3:

```
Input: n = 1
Output: 0
```

INPUT : Take input n from the user to count no of prime number less than n

OUTPUT : Gives the no of prime numbers less than n

DATA STRUCTURE USED : List

REASON : I used List to store prime status of each number (True / false) and also gives accessing number based on index

LOGIC : The program creates a boolean list and initially assumes all numbers are prime except 0 and 1. It marks all multiples of each prime number as non-prime using the Simple sieve method. Finally, it counts the remaining prime numbers in the list and prints the total number of primes less than n.

CODE:

```
def count_primes(n):
    # Create a list to mark prime numbers
    prime = [True] * n
    if n > 0:
        prime[0] = False
    if n > 1:
        prime[1] = False
```

```
    # Mark multiples as not prime
    for i in range(2, n):
        if prime[i]:
            for j in range(i * 2, n, i):
                prime[j] = False
    # Count prime numbers
    count = 0
    for i in range(n):
        if prime[i]:
            count += 1
    print("Number of primes:", count)
# Input
n = int(input("Enter n: "))
count_primes(n)
```

OUTPUT:

TEST CASE 1

```
Enter n:  10
Number of primes: 4
```

TEST CASE 2

```
Enter n:  1
Number of primes: 0
```

```
def count_primes(n):
    # Create a list to mark prime numbers
    prime = [True] * n
    if n > 0:
        prime[0] = False
    if n > 1:
        prime[1] = False
    # Mark multiples as not prime
    for i in range(2, n):
        if prime[i]:
            for j in range(i * 2, n, i):
                prime[j] = False
    # Count prime numbers
    count = 0
    for i in range(n):
        if prime[i]:
            count += 1
    print("Number of primes:", count)
# Input
n = int(input("Enter n: "))
count_primes(n)
```

```
Enter n:  10
Number of primes: 4
```

QUESTION 3 : Kth LARGEST ELEMNT IN AN LIST

215. Kth Largest Element in an Array

Medium | Topics | Companies

Given an integer array `nums` and an integer `k`, return *the* k^{th} *largest element in the array*.

Note that it is the kth largest element in the sorted order, not the kth distinct element.

Can you solve it without sorting?

Example 1:

```
Input: nums = [3,2,1,5,6,4], k = 2
Output: 5
```

Example 2:

```
Input: nums = [3,2,3,1,2,4,5,5,6], k = 4
Output: 4
```

Constraints:

- $1 <= k <= nums.length <= 10^5$
- $-10^4 <= nums[i] <= 10^4$

INPUT : Take list of integers

OUTPUT : Gives the kth Largest element (List)

DATA STRUCTURE USED : List

REASON : List have remove() to remove the current largest value in the list.

LOGIC : The program repeatedly finds and removes the largest element from the array (k−1) times.After removing the first (k−1) largest elements, the largest remaining element becomes the kth largest element.Finally, it returns and prints this remaining largest element.

CODE:

```
def find_kth_largest(nums, k):
    for i in range(k-1):
        max_num = max(nums)          # Find largest element
        nums.remove(max_num)         # Remove it
    return max(nums)                 # Remaining largest is kth largest
# Input
nums = list(map(int, input("Enter array: ").split()))
```

k = int(input("Enter k: "))
Output
print("Kth Largest Element:", find_kth_largest(nums, k))

OUTPUT:

TESTCASE 1

```
Enter array:  1 2 3 4
Enter k:  2
Kth Largest Element: 3
```

TEST CASE 2

```
Enter array:  5 6 7 3 2
Enter k:  4
Kth Largest Element: 3
```

```
def find_kth_largest(nums, k):
    for i in range(k-1):
        max_num = max(nums)          # Find largest element
        nums.remove(max_num)         # Remove it
    return max(nums)                 # Remaining largest is kth largest
# Input
nums = list(map(int, input("Enter array: ").split()))
k = int(input("Enter k: "))
# Output
print("Kth Largest Element:", find_kth_largest(nums, k))
```

```
Enter array:  5 6 7 3 2
Enter k:  4
Kth Largest Element: 3
```

QUESTION 4 : SLIDING WINDOW MAXIMUM

239. Sliding Window Maximum

Hard | Topics | Companies | Hint

You are given an array of integers `nums`, there is a sliding window of size `k` which is moving from the very left of the array to the very right. You can only see the `k` numbers in the window. Each time the sliding window moves right by one position.

Return *the max sliding window*.

Example 1:

```
Input: nums = [1,3,-1,-3,5,3,6,7], k = 3
Output: [3,3,5,5,6,7]
Explanation:
Window position                Max
---------------               -----
[1  3  -1] -3  5  3  6  7       3
 1 [3  -1  -3] 5  3  6  7       3
 1  3 [-1  -3  5] 3  6  7       5
 1  3  -1 [-3  5  3] 6  7       5
 1  3  -1  -3 [5  3  6] 7       6
 1  3  -1  -3  5 [3  6  7]      7
```

Example 2:

```
Input: nums = [1], k = 1
Output: [1]
```

INPUT : Gets the list of numbers and k window size from user

OUTPUT : A list containing the maximum element from each sliding window.

DATA STRUCTURE USED : List

REASON : Stores the input array and the maximum values of each window and allows easy slicing and index-based access to the window elements

LOGIC : The program moves a window of size k across the array from left to right.For each window, it finds the maximum element and stores it in the result list.After processing all windows, it returns and prints the list of maximum values.

CODE:

```
def sliding_window_max(nums, k):
    result = []                 # Stores maximum of each window
    for i in range(len(nums) - k + 1):
        window = nums[i:i + k]      # Current window
        result.append(max(window))  # Maximum in the window
    return result
# Input
nums = list(map(int, input("Enter array: ").split()))
k = int(input("Enter window size: "))
# Output
print("Sliding Window Maximum:", sliding_window_max(nums, k))
```

OUTPUT :

```
Enter array:  1 3 -1 -3 5 3 6 7
Enter window size:  3
Sliding Window Maximum: [3, 3, 5, 5, 6, 7]
```


```
def sliding_window_max(nums, k):
    result = []                 # Stores maximum of each window
    for i in range(len(nums) - k + 1):
        window = nums[i:i + k]      # Current window
        result.append(max(window))  # Maximum in the window
    return result
# Input
nums = list(map(int, input("Enter array: ").split()))
k = int(input("Enter window size: "))
# Output
print("Sliding Window Maximum:", sliding_window_max(nums, k))
```

```
Enter array:  1 3 -1 -3 5 3 6 7
Enter window size:  3
Sliding Window Maximum: [3, 3, 5, 5, 6, 7]
```

QUESTION 5 : REMOVE DUPLICATE LETTERS

316. Remove Duplicate Letters

Medium | Topics | Companies | Hint

Given a string `s`, remove duplicate letters so that every letter appears once and only once. You must make sure your result is **the smallest in lexicographical order** among all possible results.

Example 1:

```
Input: s = "bcabc"
Output: "abc"
```

Example 2:

```
Input: s = "cbacdcbc"
Output: "acdb"
```

INPUT : Take input string from user

OUTPUT : A string with duplicate letters removed in the smallest lexicographical order.

DATA STRUCTURE USED : List,Set,Dictionary

REASON : List stores the result characters, Set tracks added characters, and Dictionary stores the last position of each character.

LOGIC : The program stores the last occurrence of each character and processes the string one character at a time.It uses a stack to build the result, removing larger characters if they appear again later to maintain the smallest lexicographical order.Finally, it joins the stack characters and returns the string with each letter appearing only once.

CODE:

```
def remove_duplicate_letters(s):
    last = {}                # Store last index of each character
    for i in range(len(s)):
        last[s[i]] = i
    stack = []               # Store result characters
    visited = set()          # Track added characters
    for i in range(len(s)):
        ch = s[i]
        if ch in visited:
            continue
        while stack and ch < stack[-1] and last[stack[-1]] > i:
            visited.remove(stack.pop())

        stack.append(ch)
        visited.add(ch)
```

```
    return "".join(stack)
# Input
s = input("Enter string: ")
# Output
print("Result:", remove_duplicate_letters(s))
```

OUTPUT:

TEST CASE

```
Enter string:  bcabc
Result: abc
```

```
def remove_duplicate_letters(s):
    last = {}                 # Store last index of each character
    for i in range(len(s)):
        last[s[i]] = i
    stack = []                # Store result characters
    visited = set()           # Track added characters

    for i in range(len(s)):
        ch = s[i]
        if ch in visited:
            continue
        while stack and ch < stack[-1] and last[stack[-1]] > i:
            visited.remove(stack.pop())

        stack.append(ch)
        visited.add(ch)
    return "".join(stack)
# Input
s = input("Enter string: ")
# Output
print("Result:", remove_duplicate_letters(s))
```

```
Enter string:  bcabc
Result: abc
```